

Paper Presentation: MegaBlocks: Efficient Sparse Training with Mixture-of-Experts

Eungseo(Ngseo) Kim

Seoul National University, Biointelligence Lab

November 3, 2025

Original Paper Information

Original Authors

Trevor Gale¹, Deepak Narayanan², Cliff Young³, Matei Zaharia¹

Publication

MLSys 2023 - Proceedings of Machine Learning and Systems

Affiliations

¹ Stanford University, Stanford, California, USA

² Microsoft Research, Redmond, Washington, USA

³ Google Research, Mountain View, California, USA

Table of Contents

- 1 TL;DR
- 2 Key Concepts
- 3 Prior Work
- 4 Idea
- 5 Implementation Details
- 6 Experimental Results
- 7 Discussion
- 8 Additional References

1. TL;DR

The Problem

- Sparsity enables efficient scaling through conditional computation
- MoE leverages sparsity by routing tokens to specific experts
- Dynamic routing leads to uneven token distribution across experts
- Existing approach: Drop excess tokens, pad insufficient experts

MegaBlocks Solution

- Never drop tokens!
- Adapt block-sparse operations
- No performance loss, significant Expert Parallelism speedup

2. Key Concepts Overview

Key Concepts to be Covered

- **Sparsity:** A concept that activates only necessary components through conditional computation for computational efficiency
- **MoE (Mixture-of-Experts):** An architecture that combines multiple expert models to scale model capacity efficiently
- **Dynamic Routing:** A mechanism that dynamically assigns each token to appropriate experts
- **Expert Parallelism:** A technique that distributes multiple experts across different devices or processors for efficient parallel computation

2. What is Sparsity?

Conditional Computation

Sparsity uses the idea of **conditional computation**. While in dense models all parameters are used for all inputs, sparsity allows us to only run some parts of the whole system.

Scaling Without Computation Overhead

Shazeer's work showed that conditional computation allows scaling model size without increasing computation proportionally—thousands of experts can be used in each MoE layer.

Challenges

- **Uneven batch sizes:** If 10 tokens are batched, 5 might go to one expert and 5 to different experts
- **Underutilization:** This leads to uneven batch sizes and inefficient hardware usage

2. What is Mixture-of-Experts (MoE)?

Definition

MoE is an architecture that combines multiple expert models to achieve high capacity efficiently by routing each input to a small subset of experts. MoE leverages sparsity by routing tokens to specific experts, enabling conditional computation without activating all parameters.

Key Components

- **M experts:** neural networks specializing in different sub-problems
- **Router:** decides which experts to consult for each input
- **Gating network:** computes mixture weights for experts; typically linear

2. MoE Architecture Details

Mathematical Formulation

For each token x (or batch X), MoE routing works as follows
[Shazeer et al., 2017, Fedus et al., 2022]:

- Router computes logits:
 $z_m = x^T \theta^{(m)}$ for each expert
 $m \in [M]$, where $\theta^{(m)} \in \mathbb{R}^d$ is the router parameter for expert m
- Gate probabilities: $p = \text{softmax}(z)$
- Top-k selection: choose k experts with highest probabilities
- Expert networks $\{\phi_m\}_{m=1}^M$ (typically FFNs) process routed tokens
- Output: weighted combination of selected experts

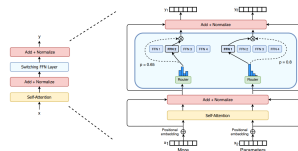


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = “More” and x_2 = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Figure 1: Switch Transformer architecture.

2. MoE Architecture in MegaBlocks

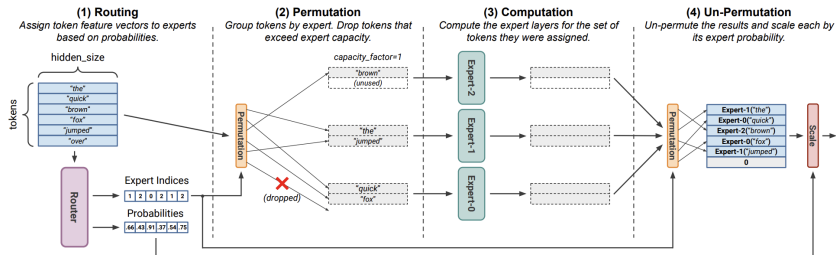


Figure 1. A Mixture-of-Experts Layer. Shown for $\text{num_experts}=3$, $\text{top_k}=1$ and $\text{capacity_factor}=1$ with the prevalent, token dropping formulation. **First (1)**, tokens are mapped to experts by the router. Along with expert assignments, the router produces probabilities that reflect the confidence of the assignments. **Second (2)**, the feature vectors are permuted to group tokens by expert assignment. If the number of tokens assigned to an expert exceeds its capacity, extra tokens are dropped. **Third (3)**, the expert layers are computed for the set of tokens they were assigned as well as any padding needed for unused capacity. **Lastly (4)**, the results of the expert computation are un-permuted and weighted by the router probabilities. The outputs for dropped tokens are shown here set to zero.

Figure 2: MoE architecture introduced by MegaBlocks.

2. So Why Dynamic?

What is Dynamic Routing?

Dynamic routing is a mechanism where the set of selected experts depends on the input. Different inputs are routed to different subsets of experts based on their characteristics.

Key Insight

When input x changes, the set of experts selected also changes:

- Different tokens x_i and x_j may be routed to completely different expert subsets
- This enables **input-adaptive computation**: experts specialize in different patterns
- Some experts may handle syntax, others semantics—depending on the token

2. Expert Parallelism

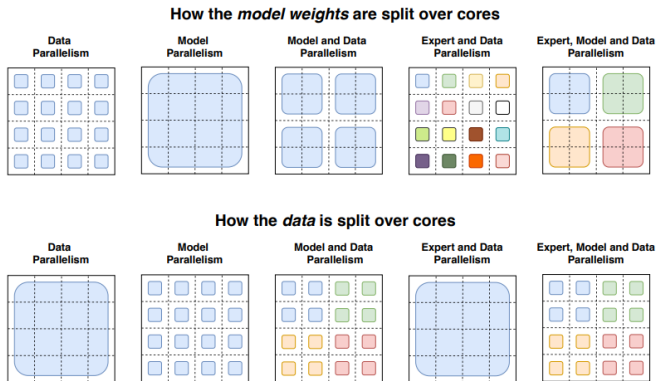


Figure: Expert parallelism architecture [Fedus et al., 2022].

3. Prior MoE Transformer Training

Fixed Expert Capacity Constraint

Prior work defines **fixed expert capacity**

[Fedus et al., 2022]:

- **Token dropping:** Excess tokens dropped
- **Padding:** Insufficient experts padded
- Static buffer allocation → inefficiency

Expert Capacity Formula

$$\text{Expert Capacity} = \frac{\text{num tokens}}{\text{num experts}} \times \text{capacity factor}$$

- Capacity factor: multiplier on uniform distribution
- Tradeoff: computation vs. quality
- Higher factor → less dropping, more padding

(A) Batched Matrix Multiplication
Compute a set of independent matrix multiplications of the same size in parallel.

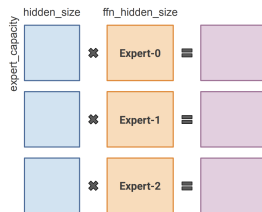


Figure: Expert capacity in Switch Transformer.

3. Expert Capacity Factor

Impact of Token Dropping

Token dropping significantly impacts model quality:

- Capacity factor = 1: 0.15 validation loss reduction
- Avoiding token dropping: 0.26 reduction (**1.73× larger**)
- Higher capacity enables better performance than dense models

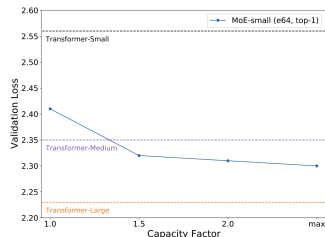
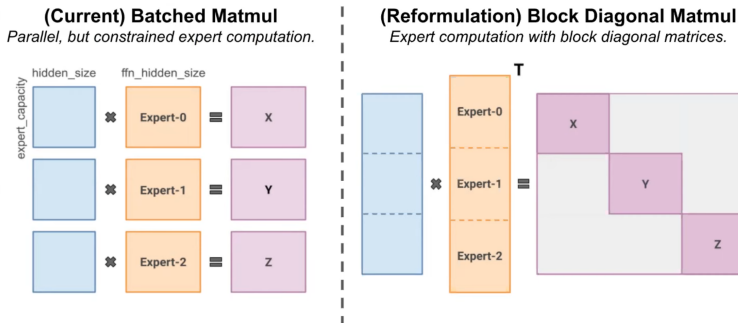


Figure 2. MoEs Trained on The Pile with Different Capacity Factors. The loss reached by the MoE models decreases significantly as expert capacity is increased, but at the cost of additional computation. The lowest loss is achieved by the “max” capacity factor model, which avoids dropping tokens through the dynamic capacity factor mechanism proposed by [Hwang et al. \(2022\)](#).

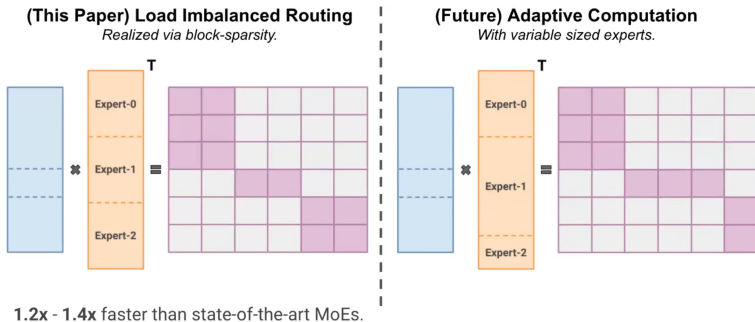
Figure: Effect of capacity factor on model quality.

4. Idea: Block Diagonal Reformulation (1)



MegaBlocks reformulates MoE computation as block-diagonal matrix operations, enabling efficient GPU implementation without token dropping.

4. Idea: Block Diagonal Reformulation (2)



Handling load-imbalanced routing through block-sparse operations allows dynamic token assignment without capacity constraints.

4. Recap: Sparse Matrix Product Notation

Notation System

Borrowed from Triton (Tillet et al., 2019) to describe sparse/dense matrix multiplication operations

Three-Character String Format

- Each character is either **S** (Sparse) or **D** (Dense)
- Order: **Output** $\rightarrow$ **Left Input** $\rightarrow$ **Right Input**

Examples

- **SDD**: Sparse output, Dense $\times$ Dense inputs
 - Also known as Sampled Dense-Dense Matrix Multiplication (SDDMM)
- **DSD** vs **DDS**: Different forms of Sparse Matrix-Dense Matrix Multiplication (SpMM)
- **SDD^T**: SDD with transposed right-hand input

4. dMoE (dropless MoE) Forward Pass Pseudocode

MegaBlocks dMoE Implementation

```
1  # x.shape: (num_tokens, hidden_size)
2  def dmoe_forward(self, x):
3      # (1) Assign tokens to experts.
4      # indices.shape: (num_tokens)
5      # weights.shape: (num_tokens)
6      indices, weights = router(x)
7
8      # (2) Create the sparse matrix topology.
9      # This describes the matrix in Figure of Block Diagonal Reformulation slide.
10     topology = make_topology(indices)
11
12     # (3) Permute the tokens to group by expert.
13     x = padded_gather(x, indices)
14
15     # (4): Compute the expert layers.
16     # inner_dim = ffn_hidden_size * num_experts
17     # self.w1.shape: (hidden_size, inner_dim)
18     # self.w2.shape: (inner_dim, hidden_size)
19     x = sdd(x, self.w1, topology)
20     x = dsd(x, self.w2)
21
22     # (5) Un-permute the tokens and scale.
23     x = padded_scatter(x, indices)
24     return x * weights
```

4. dMoE (dropless MoE) Expert Computation

Expert Layer Computation Details

- SDD (Sparse-Dense-Dense) operation for the first layer:
 $x = \text{SDD}(x, W_1, \text{topology})$
- DSD (Dense-Sparse-Dense) operation for the second layer: $x = \text{DSD}(x, W_2)$

Multi-Layer Expert Computation

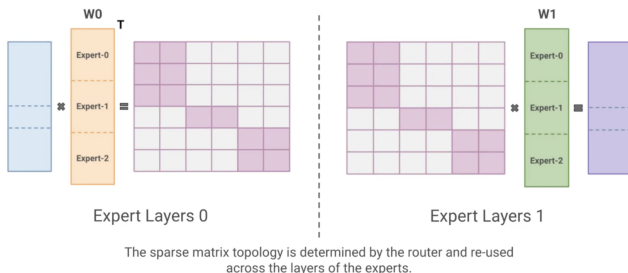


Figure: Expert computation in dMoE using block-sparse operations.

4. Efficient Block-Sparse Kernels for MoEs

Forward Pass

- First layer: **SDD**
- Second layer: **DSD**

Backward Pass

Second Layer:

- Data gradient: SDD^T
- Weight gradient: $\text{DS}^T \text{D}$

First Layer:

- Data gradient: DSD^T
- Weight gradient: $\text{DD}^T \text{S}$

4. Tile Size Selection Experiments

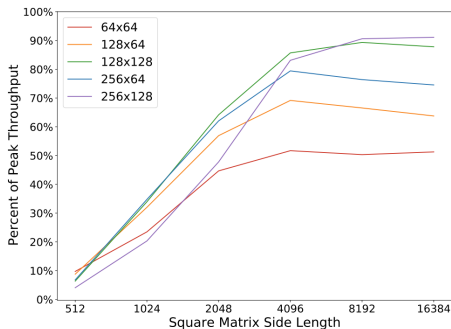


Figure 4. Matrix Multiplication Throughput with Different Tile Dimensions. Benchmarked on an A100 SXM4 80GB GPU with CUDA 11.5 and all tile dimensions supported by CUTLASS 2.5. We observe that 128x128 tiles perform consistently on-par or better than other configurations.

Figure: Performance comparison across different tile sizes. Optimal tile size balances computational efficiency and memory overhead.

4. MegaBlocks Approach

One Liner Summary

Reformulate MoE computation in terms of **block-sparse operations**

New Block-Sparse GPU Kernels

Efficiently handle the dynamism present in MoEs

Benefits

- Never drops tokens
- Maps efficiently to modern hardware
- Enables significant speedups

5. Block-Sparse Kernels for MoEs

Feature	cuSPARSE	Triton Blocksparse
Large Blocks	✓	✓
Transposition	✗	✓
Load Imbalance	✗	✓
Fast Construction	✗	✗

5. Why These Features Matter

Large Blocks

For **high throughput**: larger blocks enable better GPU utilization and performance

Transposition

For **backward pass**: transpose operations are essential for efficient gradient computation

Load Imbalance

Load imbalance: each expert processes different token counts.
For **no token dropping**: must handle unbalanced expert assignments.

Fast Construction

For **changes every use**: sparse pattern changes dynamically per forward/backward pass

5. Key Issues with Existing Libraries

cuSPARSE Limitations

- No transpose support
- ELLPACK format not suitable for dynamic patterns
 - ELLPACK: fixed-width sparse format assuming uniform row length
 - Cannot efficiently handle varying expert assignment patterns
- **Not an option for MoE training**

Triton Blocksparse Limitations

- Expensive preprocessing required
- **5–10x slower than dense if not amortized**
 - Amortized: spreading preprocessing cost over many uses
 - MoE routing changes every iteration, so no amortization possible
- Cannot reconstruct pattern efficiently every iteration

5. Their Solution: Hybrid Block Sparse Format

Kernel Design for Block-Sparse Operations

Extended NVIDIA's CUTLASS GEMM framework to directly handle block-sparse matrices.

Hybrid BCSR-COO

BCSR: Blocked Compressed Sparse Row

BCOO: Blocked Coordinate Format

Problem: SDD needs output block coordinates upfront.

Solution: Materialize row indices

- $O(1)$ coordinate lookup
- 0.024% overhead (128×128)
- Maintains row-order access

Transpose Indices

Problem: Transpose access expensive in BCSR.

Solution: Index array (no value copy)

- Points to offset in value array
- Indirect transposed access
- Minimal memory overhead

5. Hybrid Block Sparse Format Illustration

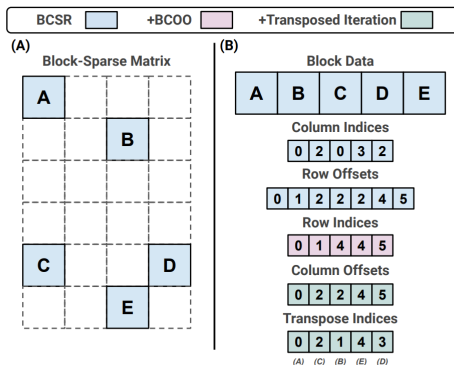


Figure 5. Block-Sparse Matrix Format used in MegaBlocks. Pane (B) shows the encoding for the sparse matrix in pane (A). Indices and offsets in our encoding are block-wise. We use blocked compressed sparse row (BCSR) as our primary sparse matrix format. We additionally store the row indices of each nonzero block (§5.1.3) and a secondary index of *transpose indices* (§5.1.4).

Figure: Hybrid blocked-CSR-COO format with transpose indices.

6. Experimental Evaluation

Three Key Experiments

This work evaluates MegaBlocks through three different experimental setups:

- ① **MoE Training Without Dropping Tokens**
- ② **MoE Training With Token Dropping**
- ③ **Block-Sparse Matrix Multiplication Performance**
 - Benchmarking the custom block-sparse kernels

6. Evaluation

Implementation

MegaBlocks is built on Megatron-LM + PyTorch.

Models

Transformers-MoEs with 64-experts and top-1 routing.

Baselines

MoE: Tutel (+ Megatron-LM)

Dense: Megatron-LM

Training

10B tokens from The Pile on 8x A100 GPUs.

Data parallelism for Transformers, 8-way expert model parallelism for MoE layers.
capacity_factor={1, 1.5, 2.0} for MoE baselines.

6. Model Configurations

Table 1: Baseline Transformer Models

Model	h_size	layers	Weights(M)	GFLOPs
XS	512	6	46	316
Small	768	12	125	879
Medium	1024	24	356	2487
Large	1536	24	760	5122
XL	2048	24	1316	8684

Table 2: Transformer-MoE Models

MoE	num_exp	top_k	Weights(M)	GFLOPs
XS	64	1	839	316
Small	64	1	3,693	879
Medium	64	1	13,041	2487

6. Experiment 1: MoE Training Without Dropping Tokens

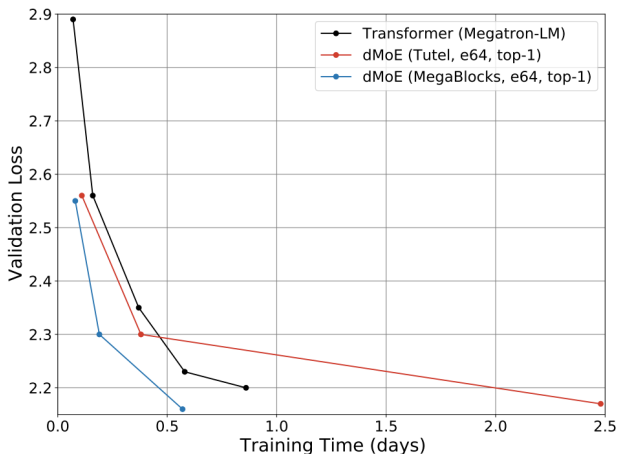


Figure 7: Performance comparison of droppless MoE training with MegaBlocks.

6. Experiment 2: MoE Training With Token Dropping

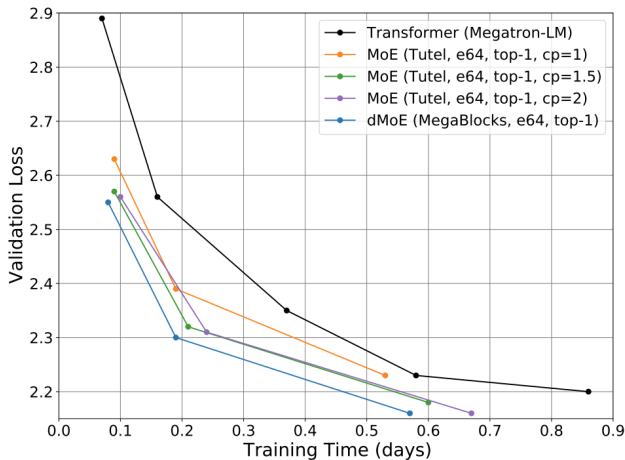


Figure 8: Comparison with traditional token-dropping approaches.

6. Experiment 3: Block-Sparse Matrix Multiplication Performance

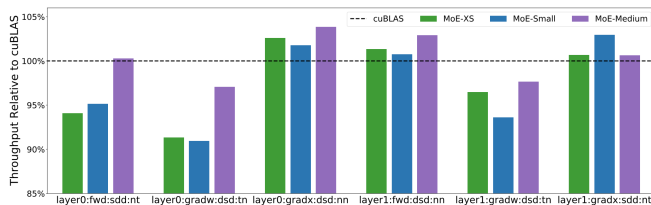


Figure 9. Block-Sparse Matrix Multiplication Throughput Compared to cuBLAS Batched Matrix Multiplication. Benchmarked for the problem configurations used in training MoE-XS, MoE-Small and MoE-Medium models. For these problems, our block-sparse matrix multiplication kernels realize 98.6% of the throughput achieved by cuBLAS on average with a standard deviation of 4% and a maximum and minimum relative throughput of 104% and 91% respectively.

Figure 9: Microbenchmark results of custom block-sparse kernels.

Observed Overhead in Weight Gradients

$DS^T D$ operation (weight gradient with transposed sparse operand) shows overhead:

- Uses **secondary index** for transpose access (no value copy/transpose)
- Weak spatial locality $\rightarrow$ reduced cache utilization $\rightarrow$ lower throughput

7. Discussion Question

Hardware Trend vs. MoE Direction

MoE models use less computation relative to their model size. However, on modern GPUs, computation is becoming relatively cheap while memory is increasingly expensive.

Key Question

This seems opposite to hardware trends—does this direction still make sense?



Fedus, W., Zoph, B., and Shazeer, N. (2022).

Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.

Journal of Machine Learning Research.

JMLR.



Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017).

Outrageously large neural networks: The sparsely-gated mixture-of-experts layer.

In *International Conference on Learning Representations*.

Thank you!
juniormoo@snu.ac.kr